

Object Oriented Programming with Java

25.3 Faculty of Computing



Take Home Activity 05

Answer all questions.

- a. Create a Java class called **HotelRoom**.
 - b. Declare four instance variables with suitable data types: **roomNumber**, **roomType**, **pricePerNight**, and **isBooked**.
 - c. Create a non-parameterized constructor that sets default values - roomNumber to 0, roomType to "Standard", pricePerNight to 5000.0, and isBooked to false.
 - d. Create a parameterized constructor that accepts all four values and assigns them to the instance variables.
 - e. Create a method called **bookRoom** that takes one parameter called nights and returns a double. If the room is not already booked, set isBooked to true and return the total cost as pricePerNight x nights. If the room is already booked, print "Room is already booked." and return 0.0.
 - f. Create a method called **checkoutRoom** that takes no parameters and has no return type. It should set isBooked to false and print "Room [roomNumber] is now available."
 - g. Create a method called **getRoomSummary** that takes no parameters and returns a String in the format: Room [roomNumber] - [roomType] - Rs. [pricePerNight] per night - Booked: [isBooked].
 - h. In the main method, create two **HotelRoom** objects using each constructor. Book one room for 3 nights and print the total cost. Try booking the same room again. Call checkoutRoom and book it again. Print the summary for both rooms.
- 2) Create a Java class called **OnlineOrder**.
- a. Declare four instance variables with suitable data types: **orderId**, **customerName**, **itemPrice**, and **quantity**.
 - b. Create a parameterized constructor that accepts all four values and assigns them to the instance variables.
 - c. Create a method called **calculateTotal** that takes one parameter called discountPercentage and returns a double. It should calculate the total as itemPrice × quantity, apply the discount, and return the final amount.
 - d. Create a method called **updateQuantity** that takes one parameter called newQuantity and has no return type. If newQuantity is greater than 0, update the quantity and print "Quantity updated to [newQuantity]." Otherwise print "Invalid quantity."
 - e. Create a method called **getOrderSummary** that takes no parameters and returns a String containing the order ID, customer name, item price, and quantity.

- f. In the main method, create two `OnlineOrder` objects. Calculate the total with a 10% discount on one. Call `updateQuantity` with a valid and an invalid value. Print the order summary for both.
- 3) Create a Java class called **StudentResult**.
- Declare five instance variables with suitable data types: **studentId**, **studentName**, **mathMark**, **scienceMark**, and **englishMark**.
 - Create a parameterized constructor that accepts all five values and assigns them to the instance variables.
 - Create a method called **calculateAverage** that takes no parameters and returns a double. It should return the average of the three marks.
 - Create a method called **getGrade** that takes no parameters and returns a String. It should return the grade based on the average — A for 75 and above, B for 65 and above, C for 50 and above, S for 35 and above, and F for below 35.
 - Create a method called **updateMark** that takes two parameters — subject (a String) and newMark (a double) — and has no return type. Based on the subject name ("math", "science", or "english"), update the correct mark. If the subject name does not match, print "Invalid subject name."
 - Create a method called **getResultSheet** that takes no parameters and returns a String containing the student ID, name, all three marks, average, and grade.
 - In the main method, create two `StudentResult` objects. Call `updateMark` with valid and invalid subject names. Print the result sheet for both students.
- 4) Create a Java class called **VehicleRental**.
- Declare five instance variables with suitable data types: **vehicleId**, **vehicleType**, **ratePerDay**, **availableKm**, and **isAvailable**.
 - Create a non-parameterized constructor that sets suitable default values for all five variables.
 - Create a parameterized constructor that accepts all five values and assigns them to the instance variables.
 - Create a method called **rentVehicle** that takes two parameters - days (an int) and estimatedKm (a double) - and returns a String. If the vehicle is available and estimatedKm is within availableKm, mark it as unavailable and return the rental confirmation with the total cost as $\text{ratePerDay} \times \text{days}$. Otherwise return an appropriate error message.
 - Create a method called **returnVehicle** that takes one parameter called kmUsed and has no return type. It should deduct kmUsed from availableKm, set isAvailable to true, and print the updated vehicle status.
 - Create a method called **getVehicleInfo** that takes no parameters and returns a String containing all five variable values.
 - In the main method, create two `VehicleRental` objects using each constructor. Rent one, try to rent it again, return it, and rent it once more. Print the vehicle info after each major action.

- 5) Create a Java class called **LibraryBook**.
- Declare five instance variables with suitable data types: **bookId, title, author, totalCopies, and borrowedCopies**.
 - Create a non-parameterized constructor that sets suitable default values for all five variables.
 - Create a parameterized constructor that accepts bookId, title, author, and totalCopies. Set borrowedCopies to 0 inside this constructor.
 - Create a method called **borrowBook** that takes one parameter called count and returns a String. Calculate available copies as totalCopies – borrowedCopies. If count is within the available copies, increase borrowedCopies by count and return "Successfully borrowed [count] copy/copies. Remaining: [available]". Otherwise return "Only [available] copy/copies available."
 - Create a method called **returnBook** that takes one parameter called count and has no return type. If count is less than or equal to borrowedCopies, reduce borrowedCopies by count and print a confirmation. Otherwise print "Return count exceeds borrowed copies."
 - Create a method called **getAvailableCopies** that takes no parameters and returns an int representing the number of currently available copies.
 - Create a method called **getBookInfo** that takes one parameter called shelf and returns a String containing book ID, title, author, total copies, borrowed copies, and shelf location.
 - In the main method, create two LibraryBook objects using each constructor. Borrow copies, try to borrow more than available, return some, and borrow again. Print book info after each action.